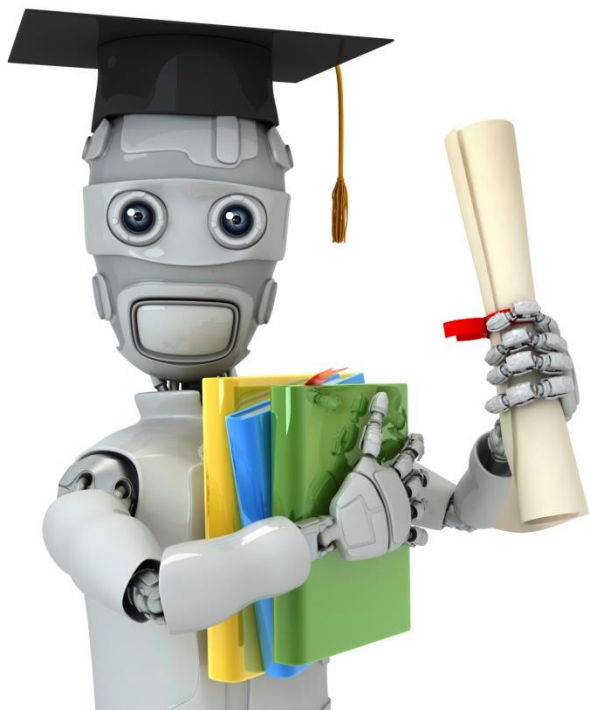




Machine Learning

Linear regression with multiple variables

Multiple features

One feature

Size (feet ²)	Price (\$1000)
2104	460
1416	232
1534	315
852	178
...	...

Multiple features

Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	40	315
852	2	1	36	178
...	...	...	...	...

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Multiple features

x_1	x_2	x_3	x_4	y
Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	40	315
852	2	1	36	178
...	...	...	...	...

n = number of features
 $x^{(i)}$ = input (features) of i th training sample
 $x_j^{(i)}$ = value of feature j in i th training sample

$$x^{(2)} = \begin{bmatrix} 1416 \\ 3 \\ 2 \\ 40 \end{bmatrix}$$

Hypothesis

Univariate linear regression

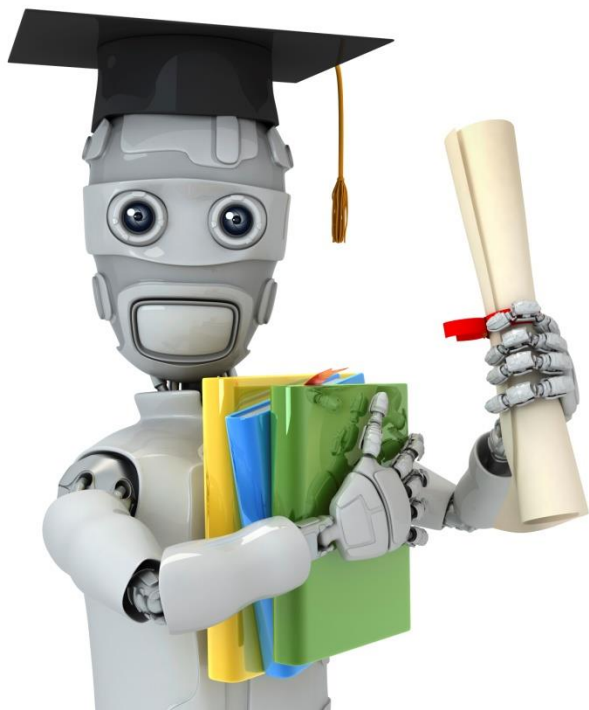
Previously:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Multivariate linear regression

$$\begin{aligned} \rightarrow h_{\theta}(x) &= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \\ &= \theta^T x \end{aligned}$$

$$\text{where } x = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n+1} \text{ and } \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} \in \mathbb{R}^{n+1}$$



Machine Learning

Linear regression with multiple variables

Gradient descent for multiple variables

Hypothesis: $h_{\theta}(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \dots, \theta_n$

Cost function:

$$J(\theta_0, \theta_1, \dots, \theta_n) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Gradient descent:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \dots, \theta_n)$$

} (simultaneously update for every $j = 0, \dots, n$)

Hypothesis: $h_{\theta}(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \dots, \theta_n$

Cost function:

$$\underbrace{J(\theta_0, \theta_1, \dots, \theta_n)}_{J(\theta)} = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Gradient descent:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} \boxed{J(\theta_0, \dots, \theta_n)} \quad J(\theta)$$

}

(simultaneously update for every $j = 0, \dots, n$)

Gradient descent

Previously (n=1):

Repeat {

$$\theta_0 := \theta_0 - \alpha \underbrace{\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial \theta_0} J(\theta)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

(simultaneously update θ_0, θ_1)

}

New algorithm ($n \geq 1$):

Repeat {

$$\theta_j := \theta_j - \alpha \underbrace{\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}}_{\frac{\partial}{\partial \theta_j} J(\theta)}$$

(simultaneously update θ_j for
 $j = 0, \dots, n$)

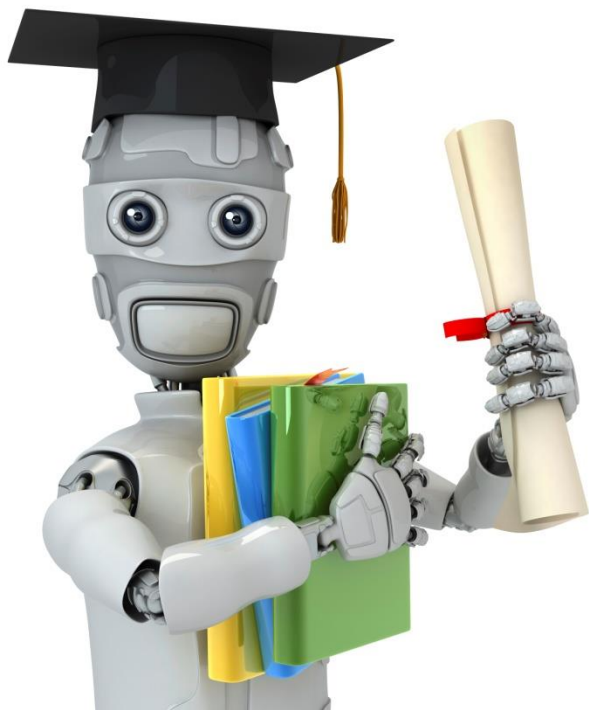
}

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_2^{(i)}$$

...



Machine Learning

Linear regression with multiple variables

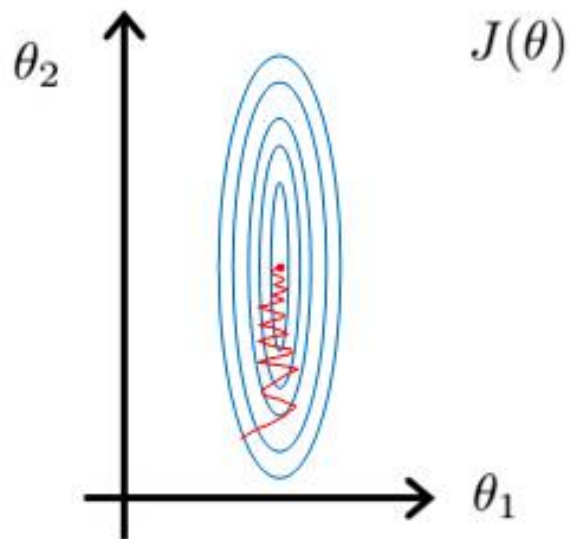
Gradient descent in practice I: Feature Scaling

Feature Scaling

Idea: Make sure features are on a similar scale.

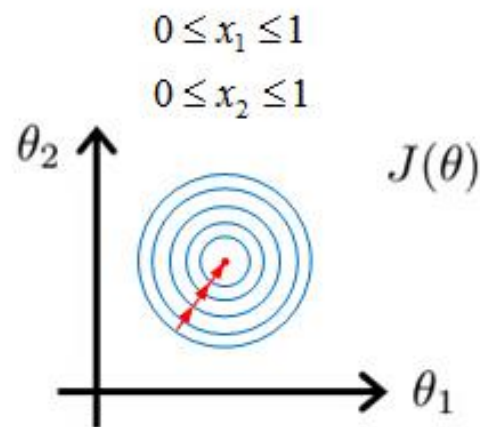
E.g. x_1 = size (0-2000 feet²)

x_2 = number of bedrooms (1-5)



$$x_1 = \frac{\text{size (feet}^2\text{)}}{2000}$$

$$x_2 = \frac{\text{number of bedrooms}}{5}$$



Feature Scaling

Get every feature into approximately a $-1 \leq x_i \leq 1$ range

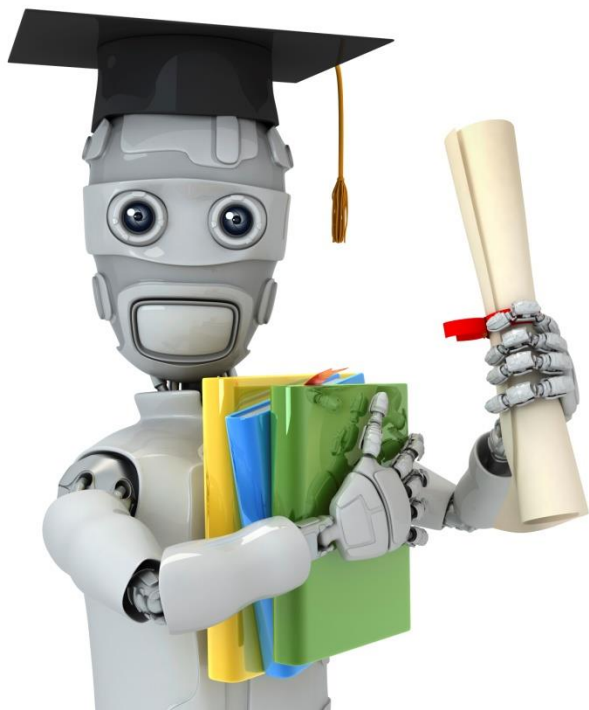
$$x_0 = 1$$

$$0 \leq x_1 \leq 3$$

$$-2 \leq x_2 \leq 0.5$$

$$-100 \leq x_3 \leq 100$$

$$-0.0001 \leq x_4 \leq 0.0001$$

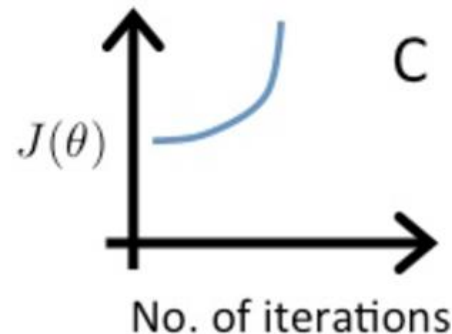
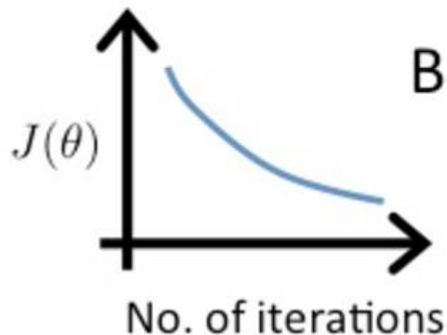
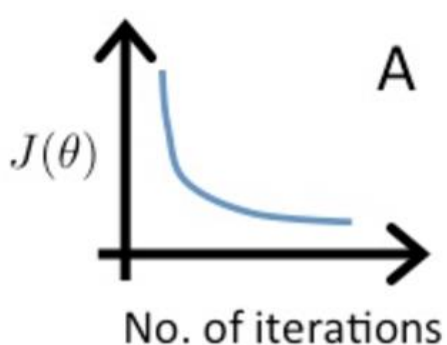


Machine Learning

Linear regression with multiple variables

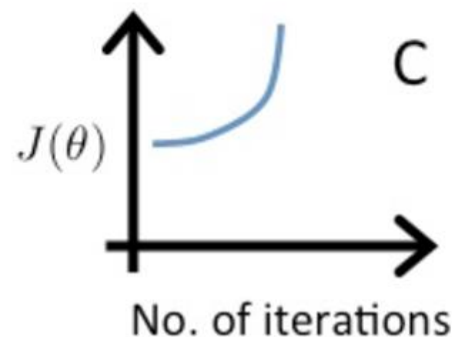
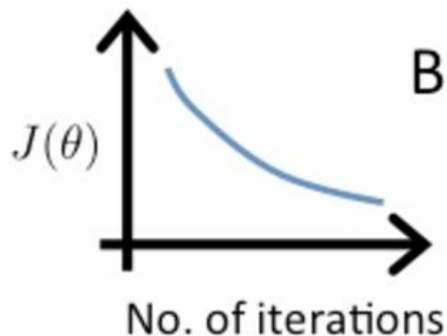
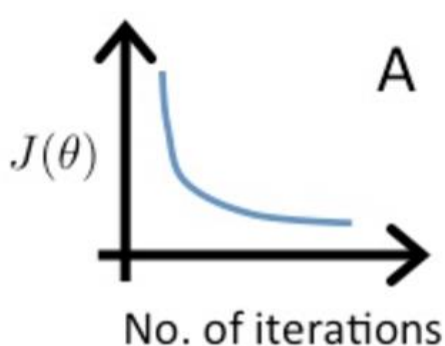
Gradient descent in practice II: Learning rate

Make sure gradient descent is working correctly.



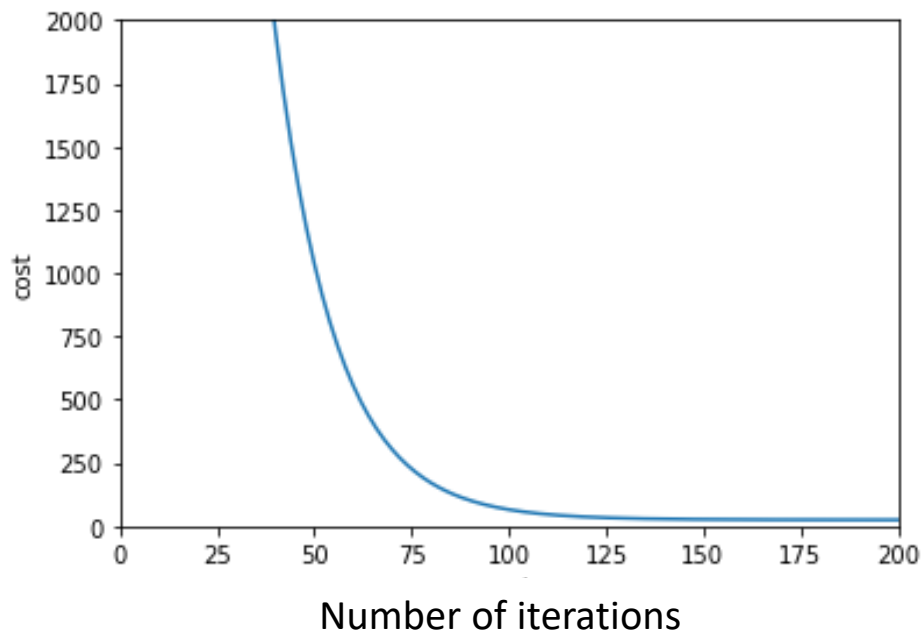
- **Plot A** is a proper learning curve.
- **Plot B** indicates the algorithm is slow to converge. It generally happens due to very small learning rate.
- **Plot C** indicates the algorithm is diverging. It generally happens if the value of learning rate α is too high.

Make sure gradient descent is working correctly.



- For sufficient small α , $J(\theta)$ should decrease on every iteration.
- If α is too small, gradient descent can be slow to converge.
- If α is too large, $J(\theta)$ may not decrease on every iteration; may not converge.

Automatic Convergence Test: gradient descent can be considered to be converged if the cost function $J(\theta)$ decreases by less than a threshold (10^{-3}) in one iteration.



In order to choose optimum value of α , run the algorithm with different values, e.g. 1, 0.3, 0.1, 0.03, 0.01, ... and plot the learning curve to understand whether the value should be increased or decreased.